TECHNICAL OVERVIEW

Orbital Voice Agent

A voice-to-voice AI assistant with hybrid retrieval-augmented generation, semantic search, agentic tool use and full observability. Architecture, flows, diagrams and the complete feature list.

STACK

NestJS 11 · Next.js 16 · Postgres 17 + pgvector · Redis + BullMQ · OpenAI

REPOSITORY

voice-rag-agent (pnpm + Turborepo monorepo)

ASSESSMENT

Senior Software Engineer · Voice-to-Voice Agent with RAG, Semantic Search & Agentic AI

DOCUMENT DATE

10 November 2026

Contents

1. Executive summary
2. Feature list
3. System architecture
4. Monorepo and layering
5. A voice turn, end to end
6. The agent loop
7. Knowledge base and retrieval
8. Tools
9. Data model
10. Contracts: REST and Socket.IO
11. Latency and cost
12. Testing and quality
13. Configuration and operations
14. Key technical decisions
15. Limitations and roadmap

1. Executive summary

The Orbital Voice Agent lets an employee talk to an assistant that answers from a company handbook, uses tools when it needs facts the handbook cannot give (dates, balances, arithmetic) or when it must act (open a ticket), and talks back. Every answer is grounded in cited passages, every step of the agent's reasoning is streamed to the screen, and every turn records its latency, token usage and estimated cost.

7 agent tools, one of them side-effecting	**12** handbook documents · 71 indexed chunks	**≈ 1.9 s** typed question to full answer (one model call)	**≈ $0.005** per spoken turn at list prices

The system is a cascaded pipeline (speech-to-text → agent → text-to-speech) rather than a single speech-to-speech model, so the retrieval, grading, tool orchestration and citations live in this codebase where they can be tested, traced and demonstrated. Latency is recovered with on-device voice activity detection, eager retrieval before the first model call, concurrent tool execution and streamed speech synthesis.

> Everything runs in two modes from the same code: **OpenAI mode** with real models when an API key is present, and a deterministic **mock mode** (scripted planner, hashing embeddings, replayed transcription, synthesized audio) that exercises the whole pipeline offline. All automated tests and CI use mock mode and never spend money.

2. Feature list

Voice experience

- **Hands-free listening** with Silero voice activity detection running in the browser (ONNX runtime, assets served same-origin); only the speech segment is uploaded as 16 kHz WAV.
- **Push-to-talk** mode using the same detector, submitting on release.
- **Barge-in**: real speech during playback cancels the current turn through an abort signal that reaches the model call, the tools and the speech stream.
- **Streamed speech**: replies are split at sentence boundaries while the model is still writing; each sentence is synthesized immediately and delivered as raw PCM chunks played gaplessly on the Web Audio clock.
- **State-driven orb**: idle, listening with level meter, transcribing, thinking, using tools, speaking.
- **Text fallback** through the same pipeline for accessibility and testing.
- **Domain vocabulary prompt** for transcription (product names, acronyms).

Knowledge base and retrieval

- **Structure-aware chunking**: markdown sections by heading, recursive paragraph/sentence splitting to ~400 tokens with sentence overlap, tiny sections merged.
- **Contextual chunk headers**: each chunk is embedded with its document title and section path.
- **Hybrid search**: pgvector cosine similarity plus a BM25 keyword channel over stored lexemes, fused with weighted reciprocal rank fusion, for up to three query phrasings.
- **Retrieval grading** (strong / weak / none) from calibrated similarity thresholds and cross-channel agreement.
- **Search playground** with per-channel scores, three modes, and a document viewer.
- **Asynchronous ingestion** on Redis + BullMQ with job progress; uploads and re-indexing from the API; checksum-based skipping.
- **Evaluation script** reporting hit@1, hit@3 and MRR for paraphrased questions in every mode.

Agent and tools

- **ReAct-style loop** with a step budget; tool failures are fed back as structured errors the model can recover from.
- **Eager retrieval**: passages for the message are fetched before the first model call, so most questions finish in one round.
- **Concurrent tool execution** with per-tool timeouts and cancellation.
- **Seven tools**: knowledge search, current date/time, business-day counting, calculator, leave balance, ticket lookup through a ticketing API, support ticket creation.
- **Confirmation before side effects**: the agent offers to open a ticket and acts only on a clear yes.
- **Citations** renumbered in order of appearance; markers are removed from speech.
- **Memory**: token-budgeted sliding window plus a persona (name, employee id, department, timezone), so follow-ups work and nothing already known is asked again.

Observability, UX and engineering

- **Live agent trace**: tool calls with arguments, results with durations, retrieval grades, state transitions.
- **Latency badges** per turn (speech-to-text, first token, first audio, total) and **cost badges** with token counts.
- **Sources sheet** from citation chips; conversation history with "continue by voice".
- **Tools & tickets dashboard**: tool catalogue with schemas, usage statistics, support tickets with status changes, HR balances.
- **Light, dark and system themes**; keyboard and screen-reader friendly controls.
- **One contract package**: Zod schemas generate API DTOs, OpenAPI docs and client types.
- **Ports and adapters** for every AI vendor; **mock mode**; 100+ automated tests incl. a real microphone flow; CI; one-command bootstrap.

3. System architecture

The browser owns capture, playback and presentation. The API owns the session, the turn pipeline, the agent loop, retrieval, tools and persistence. OpenAI provides the models behind three ports; Postgres stores vectors, full-text indexes, conversations and tickets; Redis backs the ingestion queue.

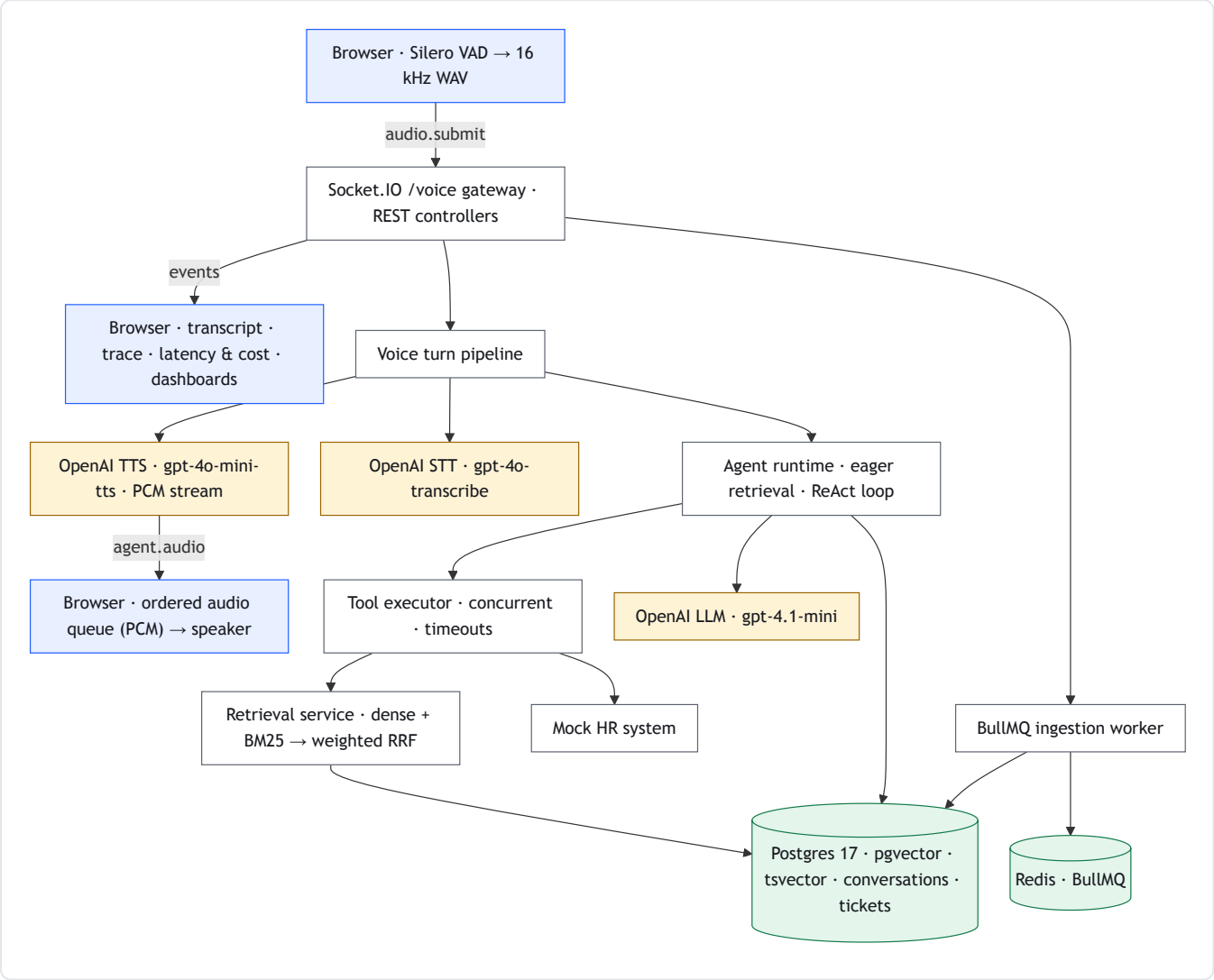


Figure 1. Runtime components and the main data paths (blue: browser, white: API, amber: OpenAI ports with mock adapters in tests, green: data stores).

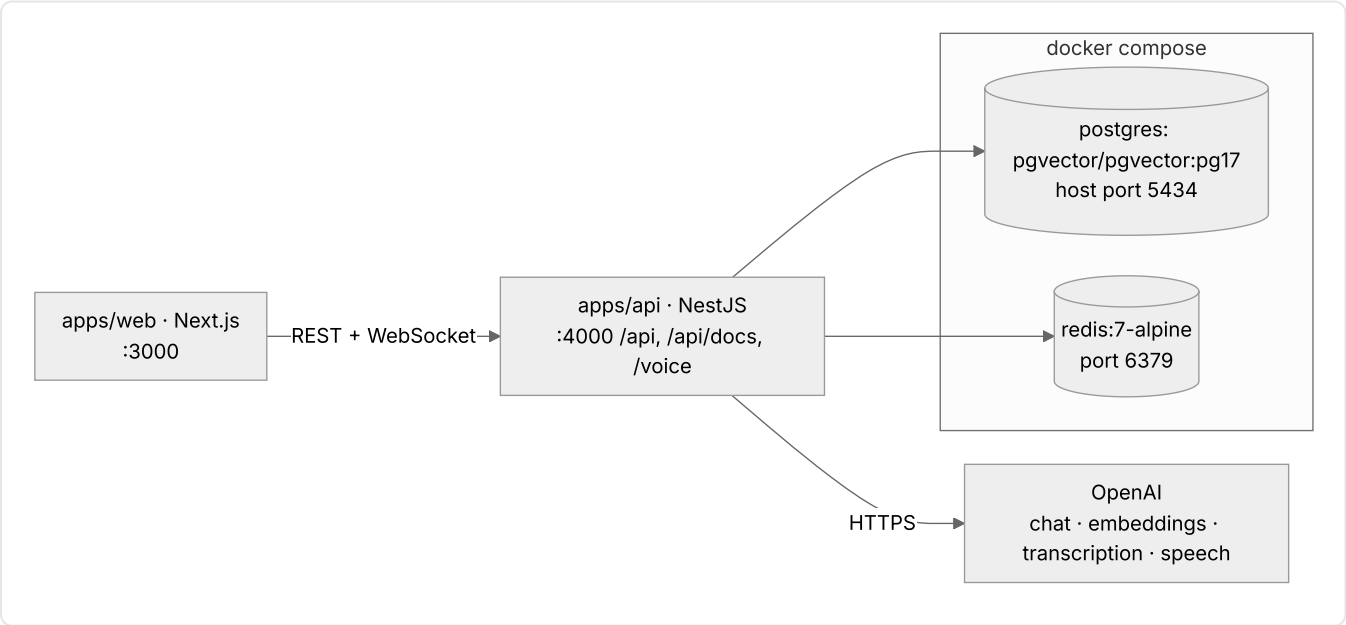


Figure 2. Local deployment. `pnpm bootstrap` starts the containers, migrates, seeds and indexes; `pnpm dev` runs both apps.

4. Monorepo and layering

```
voice-rag-agent/
├─ apps/api            NestJS: common/ (config, database, queue, logging) · modules/
(openai, embeddings, llm,
│                      speech, knowledge, hr, tools, conversations, agent, voice, health)
├─ apps/web            Next.js: app/ routes · components/ (voice, transcript, trace,
knowledge,
│                      conversations, tools, layout, ui) · features/ (voice-session,
audio, knowledge, …) · lib/
├─ packages/contracts  Zod schemas → DTO types · socket event map · tool argument schemas
· personas
├─ packages/config     shared tsconfig / ESLint bases
├─ docker/             Postgres + pgvector, Redis
└─ scripts/            bootstrap, infra checks
```

Every NestJS module has the same shape, and every folder exports through an `index.ts` barrel:

LAYER	RESPONSIBILITY	EXAMPLES
Presentation	Validate DTOs, map to and from contracts, no business logic	`KnowledgeController`, `ToolsController`, `VoiceGateway`
Application	Use-cases and orchestration	`AgentRuntime`, `VoiceTurnPipeline`, `IngestionService`, `RetrievalService`
Domain	Pure logic and ports	`DocumentChunker`, `reciprocalRankFusion`, `bm25`, `RetrievalEvaluator`, `Tool`, `LlmProvider`, `SttProvider`, `TtsProvider`
Infrastructure	Adapters and repositories	Drizzle repositories, OpenAI adapters, mock adapters, BullMQ processor

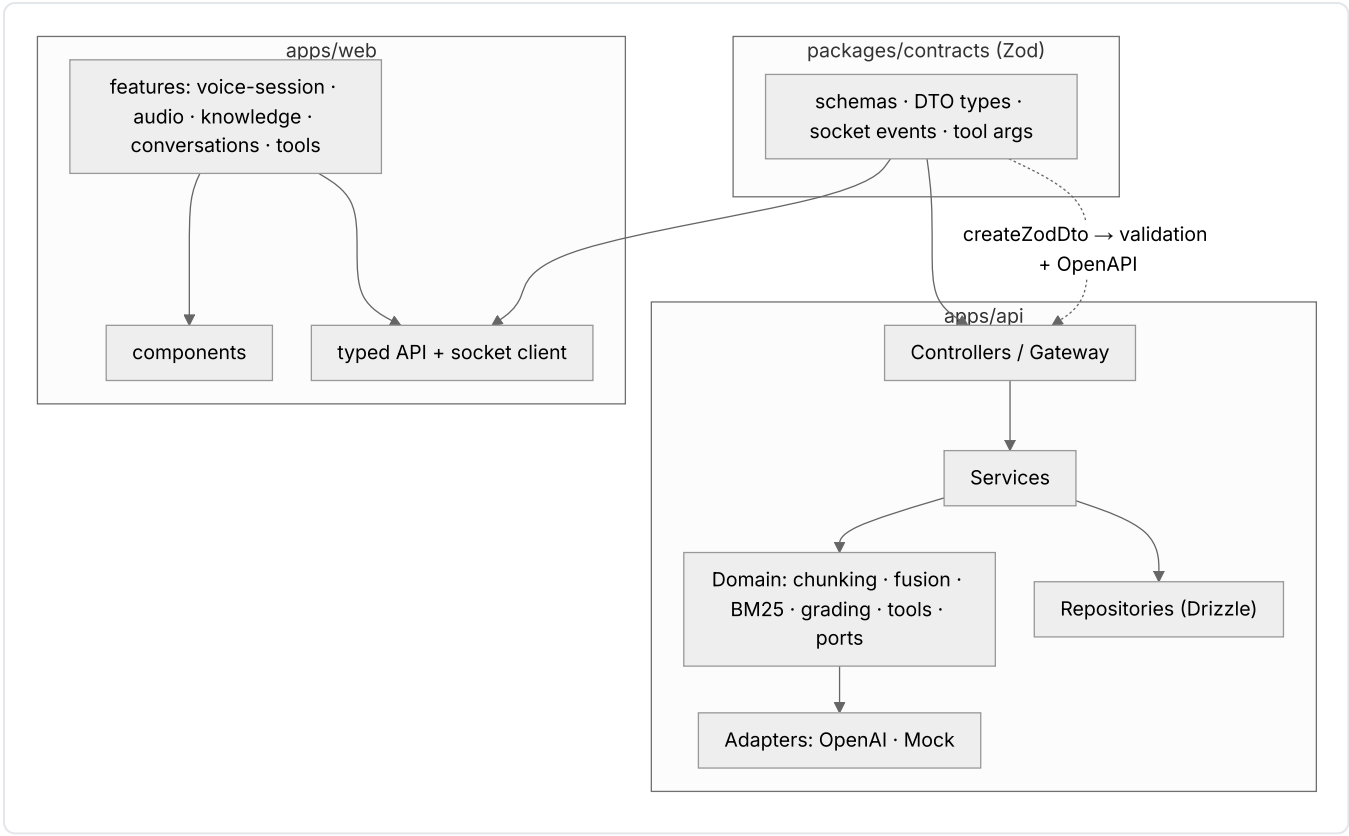


Figure 3. Layering and the single source of truth for types.

5. A voice turn, end to end

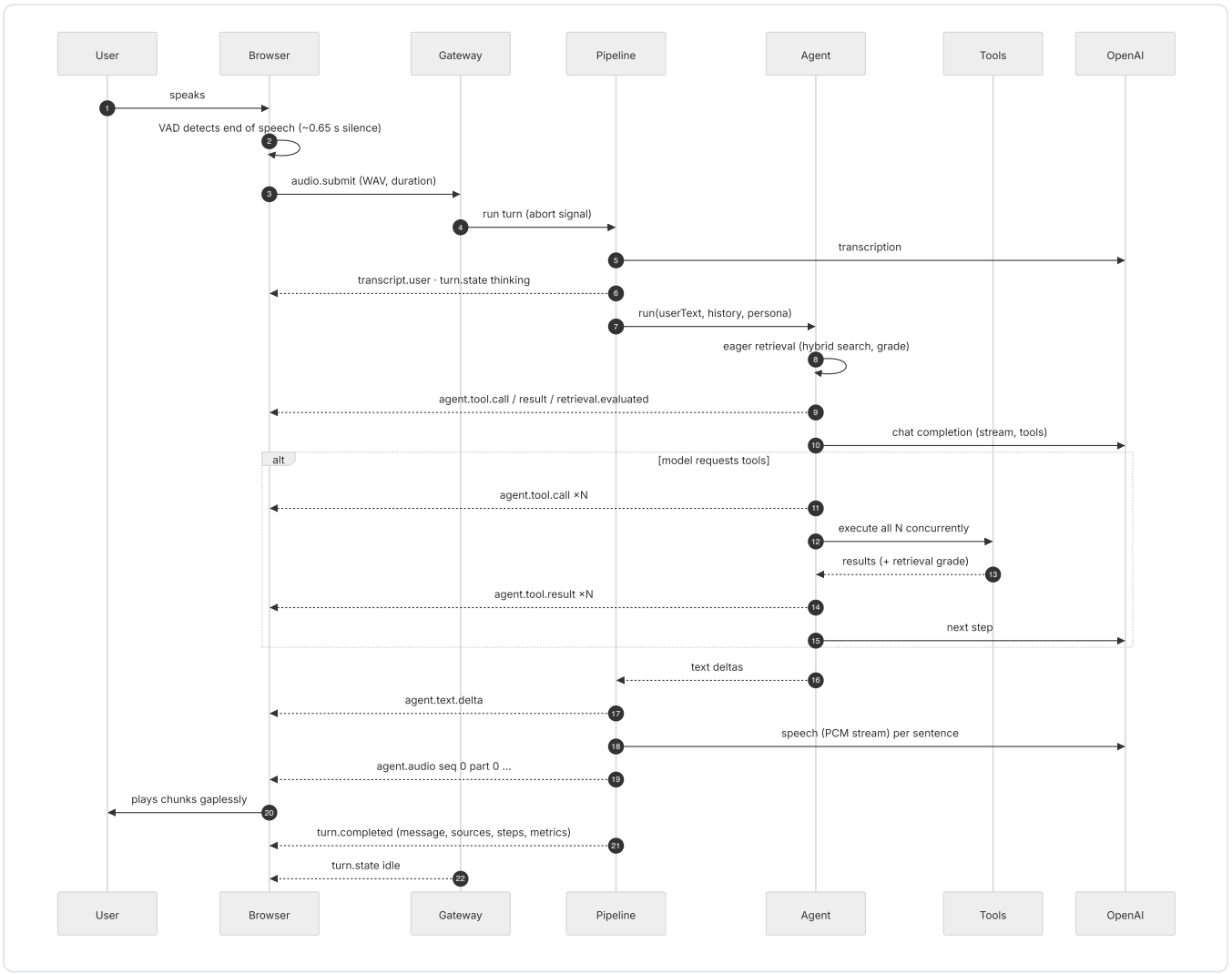


Figure 4. One spoken turn. Steps 8–9 give simple questions a single model round; steps 11–15 run only when the model asks for tools.

A typed turn follows the same path from step 7 on (through `text.submit` on the socket, or the non-streaming `POST /api/agent/turn` used by tests). Cancellation (barge-in, reset, disconnect) aborts the signal; the runtime stops the model stream, the executor stops waiting for tools, and the speech scheduler drops queued chunks.

6. The agent loop

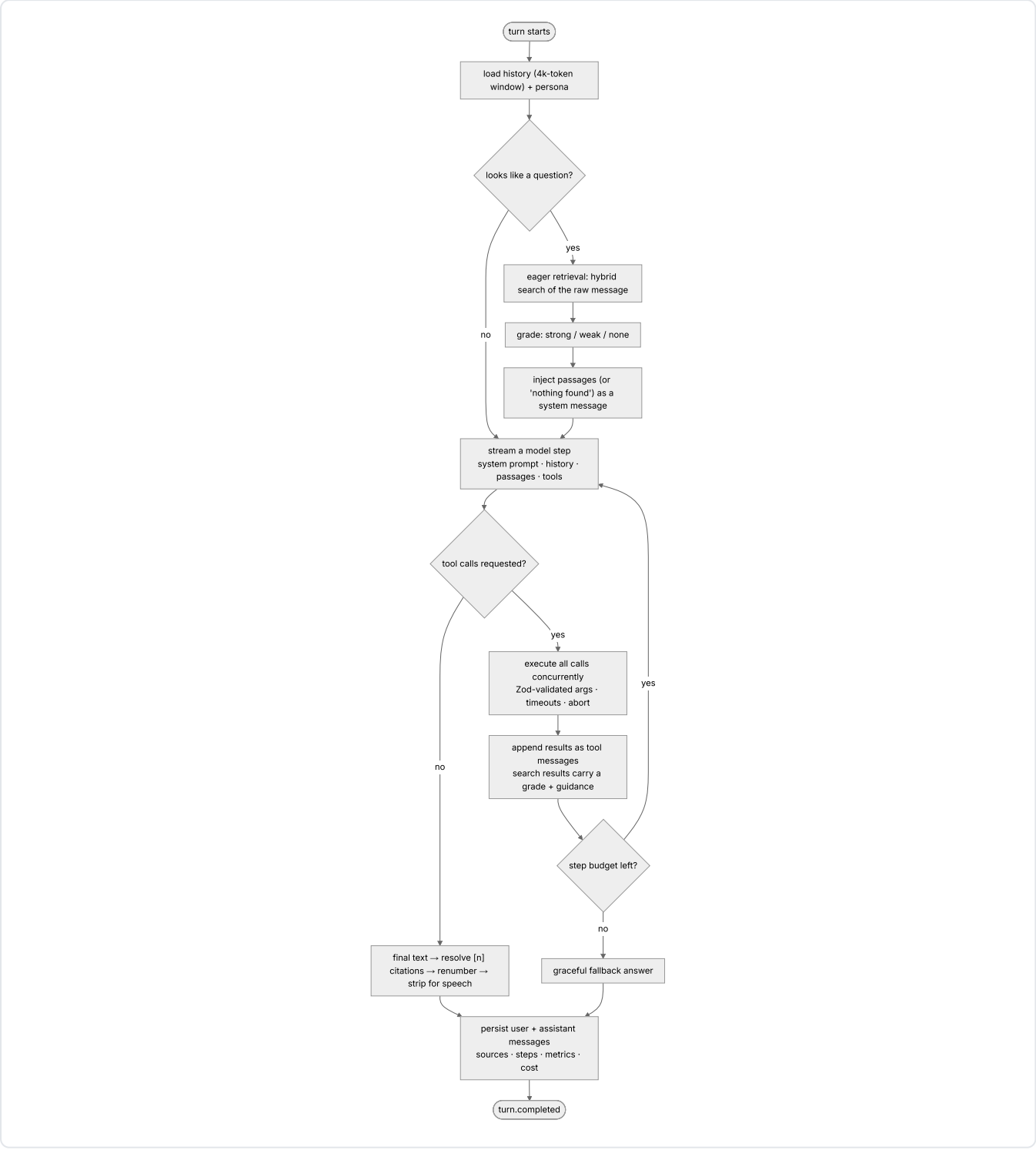


Figure 5. Control flow of `AgentRuntime.run()`. Every box emits typed events consumed by the REST endpoint and the voice gateway.

The system prompt encodes the policy: ground every policy statement in passages and cite them; reformulate once on a weak grade; say so and point to People Operations or the helpdesk on none; use tools instead of guessing dates or arithmetic; call independent tools together; offer to open a ticket when a policy says so and act on a clear yes; speak in short plain sentences with no markdown.

7. Knowledge base and retrieval

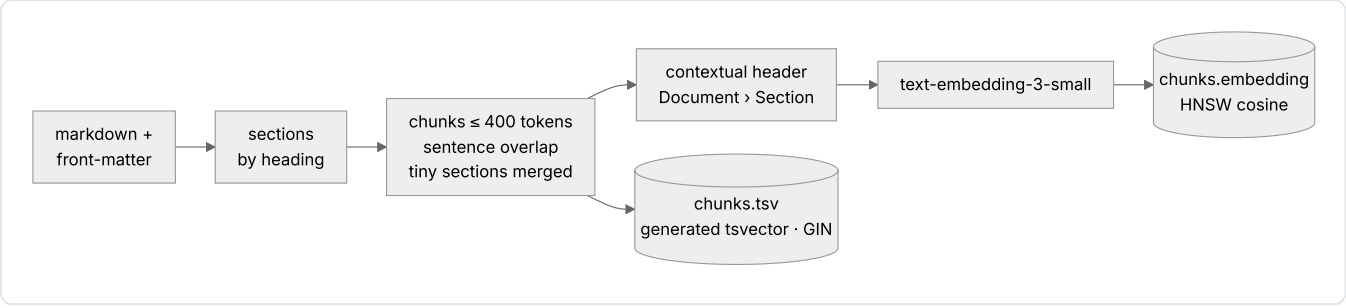


Figure 6. Ingestion: inline for the seed script, as BullMQ jobs for uploads and re-indexing. Unchanged documents are skipped by checksum.

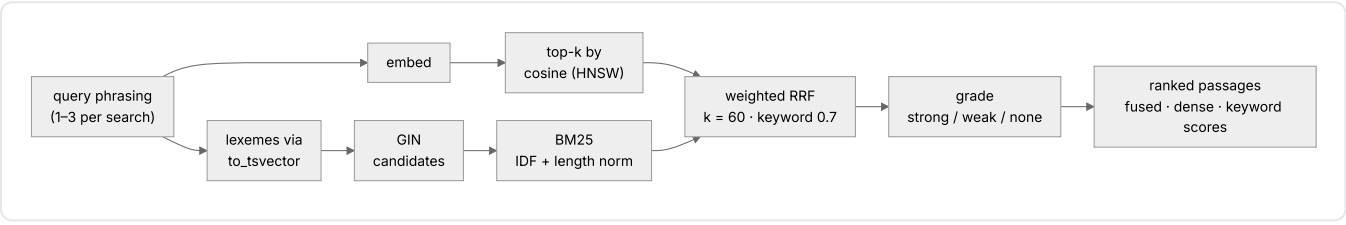


Figure 7. Hybrid retrieval: both channels run concurrently for every phrasing and read the same table.

Evaluation with nineteen paraphrased questions (`pnpm eval:retrieval`):

MODE	HIT@1	HIT@3	MRR	MOCK VECTORS HIT@1 / HIT@3 / MRR
hybrid	0.89	0.94	0.92	0.89 / 0.94 / 0.91
dense	0.94	1.00	0.96	0.72 / 0.89 / 0.81
keyword (BM25)	0.72	0.89	0.79	0.67 / 0.89 / 0.78

With real embeddings the dense channel alone is already excellent on paraphrases; hybrid matches it at hit@3 and keeps exact-term robustness (ticket references, product names, numbers). With the lexical mock vectors, fusion clearly beats either channel.

The twelve documents

Employee Handbook Overview · PTO and Leave Policy · Company Holidays 2026 · Remote and Hybrid Work Policy · Expense and Travel Policy · IT Security Policy · Equipment and Laptop Policy · Benefits Overview · New Hire Onboarding Guide · IT Helpdesk and Support Process · Code of Conduct · Performance Reviews and Promotions. Topics deliberately absent (pets, equity, relocation, visas) exercise the "insufficient information" behaviour.

8. Tools

TOOL	WHAT IT DOES	SIDE EFFECTS	WHERE ITS EFFECT IS VISIBLE
`search_knowledge_base`	Hybrid search for 1–3 phrasings; returns graded, numbered passages with guidance	none	Trace (grade, passages), citation chips, sources sheet
`get_current_datetime`	Date, weekday and time in the user's timezone	none	Trace result summary
`count_business_days`	Working days in a range minus the holidays the agent looked up	none	Trace: "9 weekdays − 3 holidays = 6 business days"
`calculate`	Sandboxed arithmetic (mathjs with code-execution functions removed)	none	Trace result summary
`get_leave_balance`	PTO and sick balance from the simulated HR system; defaults to the current persona	none	Trace; HR cards on the Tools page
`get_support_tickets`	The user's tickets with status and priority, fetched **over HTTP from the ticketing REST API** (`TICKETING_API_URL` ; this app's own `/api/hr` by default, in-process in tests)	none	Trace; matches the Tools & tickets table
`create_support_ticket`	Opens a ticket after explicit confirmation; returns a TCK-reference	writes	Tools & tickets page (status editable), `GET /api/hr/tickets`

Tools implement one interface (name, description, Zod argument schema, timeout, mutating flag, `execute`). The registry converts schemas to JSON Schema for the model and the executor validates arguments before running anything. The Tools & tickets page shows the catalogue exactly as the model sees it, aggregated usage from stored traces, the tickets table and the HR balances.

9. Data model

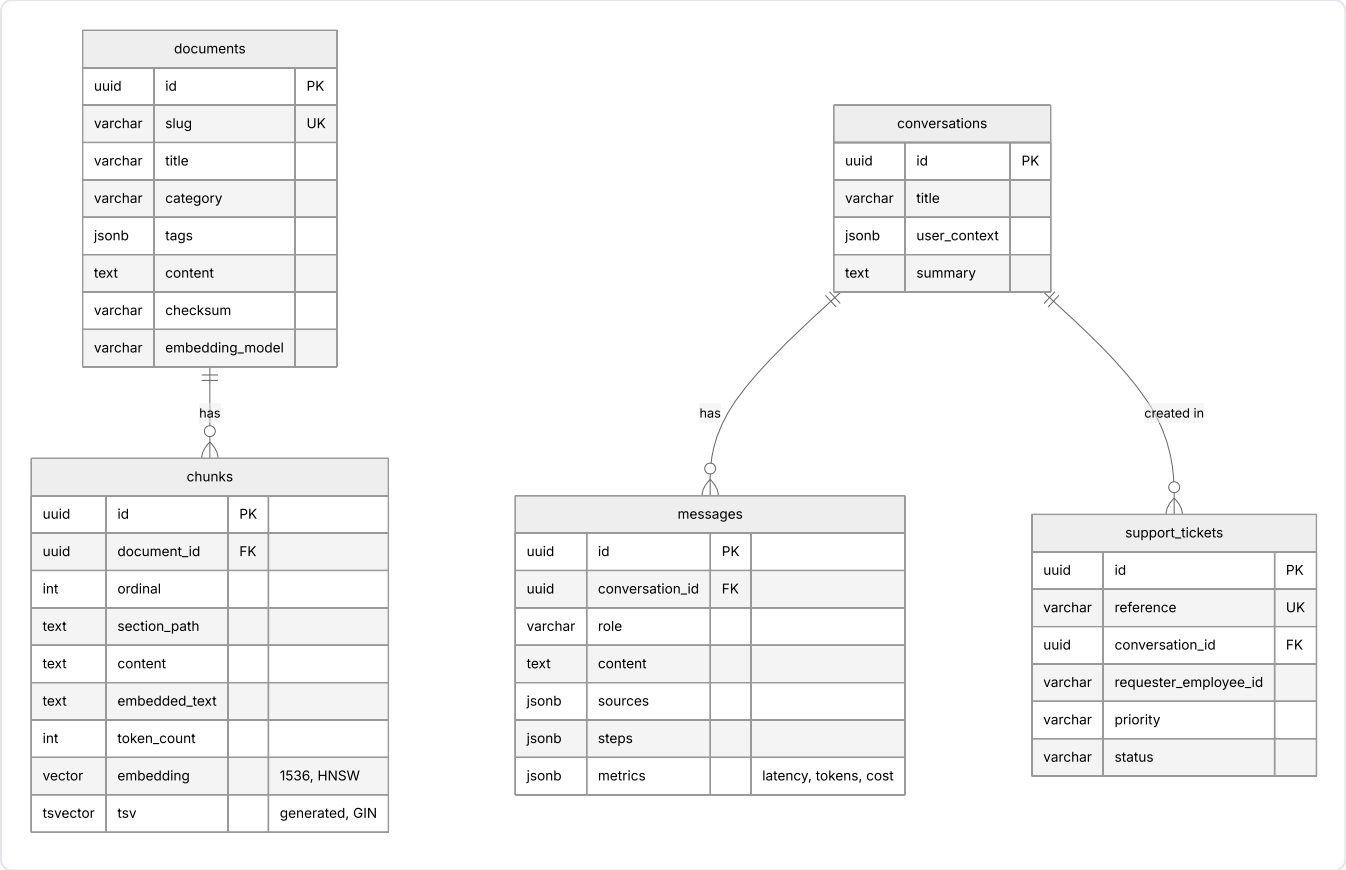


Figure 8. Postgres schema (Drizzle ORM, SQL migrations). Employees live in an in-memory mock of an external HR system.

10. Contracts: REST and Socket.IO

METHOD	PATH	PURPOSE
GET	`/api/health`	liveness, provider, dependency checks
GET	`/api/knowledge/documents[/:id]`	documents with chunk statistics; full content and sections
POST	`/api/knowledge/search`	hybrid / dense / sparse search with scores
POST	`/api/knowledge/reindex?force=` · `/api/knowledge/documents`	queue indexing jobs · upload a markdown document
GET	`/api/knowledge/jobs[/:id]`	ingestion job status
POST	`/api/agent/turn`	one non-streaming text turn
GET/POST/DELETE	`/api/conversations[/:id]`	history with transcripts, sources, traces, metrics
GET	`/api/tools` · `/api/tools/usage`	tool catalogue with schemas · usage statistics and recent invocations
GET/PATCH	`/api/hr/tickets?requesterEmployeeId=&status=&reference=` · `/api/hr/tickets/:id/status` · `/api/hr/employees[/:id/leave-balance]`	tickets created by the agent (filterable; backend of the ticket-lookup tool) · mock HR data

Socket.IO namespace `/voice` (typed in `packages/contracts`): client → server `session.start`, `audio.submit`, `text.submit`, `turn.cancel`, `conversation.reset`; server → client `session.ready`, `turn.state`, `transcript.user`, `agent.text.delta`, `agent.tool.call`, `agent.tool.result`, `agent.retrieval.evaluated`, `agent.audio` (seq, part, final, PCM), `turn.completed`, `turn.error` (with a `fatal` flag). OpenAPI documentation is served at `/api/docs`.

11. Latency and cost

STAGE	TECHNIQUE
End of speech	Silero VAD in the browser, ~650 ms trailing silence; speech segment only
Speech-to-text	`gpt-4o-transcribe` (≈1.0 s in benchmark vs 1.4–2.2 s for the mini model, +$0.003/min)
Eager retrieval	~0.3 s before the first model call; saves a whole model round for most questions
First token	non-reasoning model, compact prompt, streaming, 400-token reply cap; ~1.0–1.5 s per call incl. network
Tools	concurrent execution; dense and keyword channels in parallel inside Postgres
First audio	sentence-level pipelining; raw PCM streamed from the first ~100 ms of audio (≈0.8 s after synthesis starts)

MEASURED TURN (OPENAI MODE, ~0.3 S NETWORK RTT PER CALL)	MODEL CALLS	FIRST TOKEN	FIRST AUDIO	TOTAL	EST. COST
Typed policy question with carry-over follow-up	1	1.4 s	–	1.9 s	$0.0009
December leave: holidays + balance + day count	2	2.8 s	–	3.4 s	$0.0021
"Can I bring my dog?" → weak grade → reformulate → abstain	2	3.0 s	–	3.3 s	$0.0014
Spoken question, 6 s utterance (STT 1.3 s)	1	3.1 s	4.5 s	6.0 s	$0.005

COST COMPONENT	PRICE BASIS	TYPICAL
Chat (`gpt-4.1-mini`)	$0.40 / 1M input ($0.10 cached) · $1.60 / 1M output	$0.001–0.002 per turn
Speech-to-text (`gpt-4o-transcribe`)	$0.006 / minute	$0.0006 per 6 s utterance
Text-to-speech (`gpt-4o-mini-tts`)	≈ $0.015 / minute of speech	$0.004 per 60-word reply
Embeddings (`text-embedding-3-small`)	$0.02 / 1M tokens	negligible; whole knowledge base ≈ $0.0002

Levers that keep cost low: eager retrieval, the reply cap, a 4 000-token history window, OpenAI's automatic prompt caching (the stable prefix is billed at a quarter price), no LLM reranker by default, and mock providers in every automated test.

12. Testing and quality

SUITE	COVERS
API unit (Jest, 71)	chunking, RRF, BM25, grading, tool registry/executor, tools, mock providers, memory, citations, prompt, agent loop with a scripted model (single step, concurrent tools, eager retrieval, weak → reformulate, failure recovery, step budget, cancellation), sentence chunker, streamed speech scheduler, voice pipeline
API e2e (Jest + supertest + socket.io-client, 20)	knowledge endpoints on a real Postgres, agent turns with memory and a confirmed ticket, personas, tools and HR endpoints, the Socket.IO gateway with ordered audio
Contracts + web unit (Vitest, 13)	schema defaults and tool-name coverage, session store state machine, ordered audio queue incl. raw PCM, WAV encoder
Browser e2e (Playwright, 7)	knowledge search and viewer, agent text turns with citations and trace, multi-turn confirmed action, conversations, theme, tools dashboard, and a hands-free voice turn with Chromium's fake microphone fed a synthesized utterance

CI (GitHub Actions) runs lint, typecheck, unit, API e2e and the browser suite with Postgres and Redis service containers, all in mock mode.

13. Configuration and operations

VARIABLE	DEFAULT	EFFECT
`AI_PROVIDER`	auto	`openai` when a key is present, otherwise `mock`
`OPENAI_CHAT_MODEL` / `OPENAI_EMBEDDING_MODEL`	gpt-4.1-mini / text-embedding-3-small	reasoning model and vector model (documents remember their embedding model)
`OPENAI_STT_MODEL` / `OPENAI_TTS_MODEL` / `OPENAI_TTS_VOICE`	gpt-4o-transcribe / gpt-4o-mini-tts / coral	speech models and voice
`AGENT_MAX_STEPS` · `AGENT_MAX_OUTPUT_TOKENS` · `AGENT_EAGER_RETRIEVAL`	6 · 400 · true	loop budget, reply cap, pre-fetch
`RETRIEVAL_TOP_K` · `RETRIEVAL_SPARSE_WEIGHT` · `RERANK_ENABLED`	5 · 0.7 · false	passages per search, keyword weight in fusion, optional reranker
`TICKETING_API_URL`	this API's `/api/hr`	ticketing REST API the ticket-lookup tool calls (`inprocess` in tests)
`DATABASE_URL` · `REDIS_URL` · `DB_MIGRATE_ON_START`	compose defaults	infrastructure

Commands: `pnpm bootstrap` (idempotent setup), `pnpm dev`, `pnpm build / lint / typecheck / test`, `pnpm db:seed`, `pnpm eval:retrieval`, `pnpm --filter @voice-agent/api bench:speech`, `pnpm infra:up / infra:down / infra:reset`.

14. Key technical decisions

- **Cascaded pipeline instead of the OpenAI Realtime API.** Realtime is faster per turn but hides the agent loop, retrieval grading and citations, costs roughly ten times more and is hard to test. The providers sit behind ports, so a Realtime adapter would be an addition, not a rewrite.
- **No message broker in the request path.** A voice turn is latency-critical and single-tenant; in-process async streams are the right tool, and Node's event loop already runs independent I/O concurrently. Redis + BullMQ is used where a queue is genuinely useful: asynchronous ingestion with job status.
- **Postgres + pgvector as the single store.** Vectors, full-text search, conversations and tickets in one database with one migration path; hybrid retrieval is two indexed queries on one table.
- **BM25 computed in the application over stored lexemes.** Postgres' `ts_rank` has no rarity term; BM25 lifted hybrid hit@1 from 0.83 to 0.89 with real embeddings.
- **Eager retrieval plus retrieval-as-a-tool.** One model call for the common case, while the agent still reformulates, searches again or abstains.

- **Streamed PCM speech instead of per-clip MP3.** Every format waited 1.6–2.7 s per clip for the whole file; streaming raw PCM starts playback on the first 100 ms of audio with no client-side decoding.
- **A hand-rolled agent runtime** of a few hundred readable, tested lines rather than a framework abstraction.
- **One contract package** so nothing is defined twice, and **mock adapters as first-class citizens** so the whole system is testable without secrets.
- **NestJS 11** rather than the ten-week-old NestJS 12, because the Zod DTO library has not declared support for 12 yet.

15. Limitations and roadmap

- Mock mode demonstrates the pipeline but cannot understand arbitrary speech or questions.
- The retrieval grade is a heuristic; an LLM-based relevance grader for borderline cases is the natural next step (a reranker port already exists).
- Conversation summaries are stored but not yet generated; long conversations rely on the sliding window.
- Streaming transcription would shave a few hundred milliseconds off the first stage.
- Authentication, multi-tenancy and per-user rate limiting are out of scope for the assessment.

Appendix: screenshots

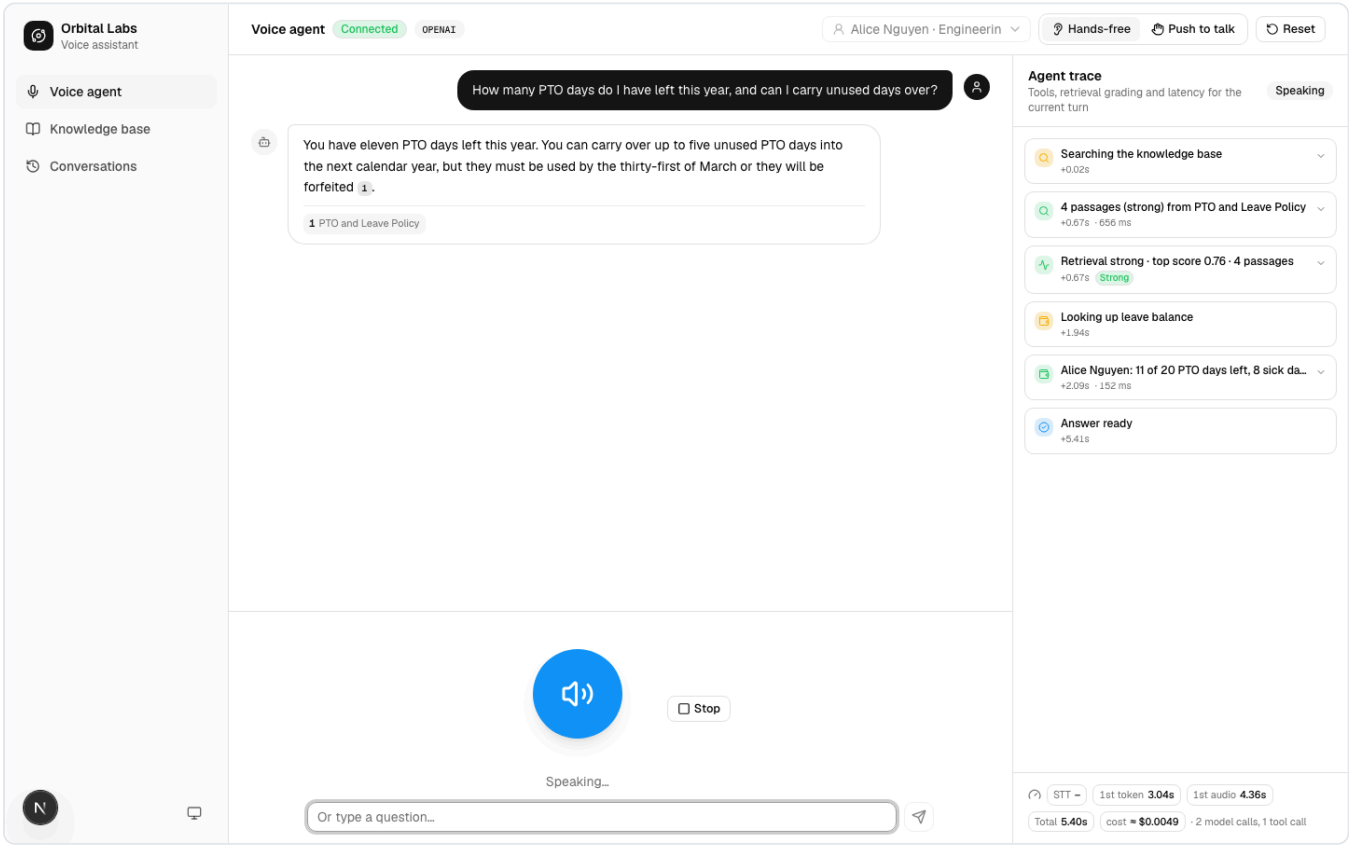


Figure 9. Voice agent: transcript with citation chips, live trace, latency and cost badges.

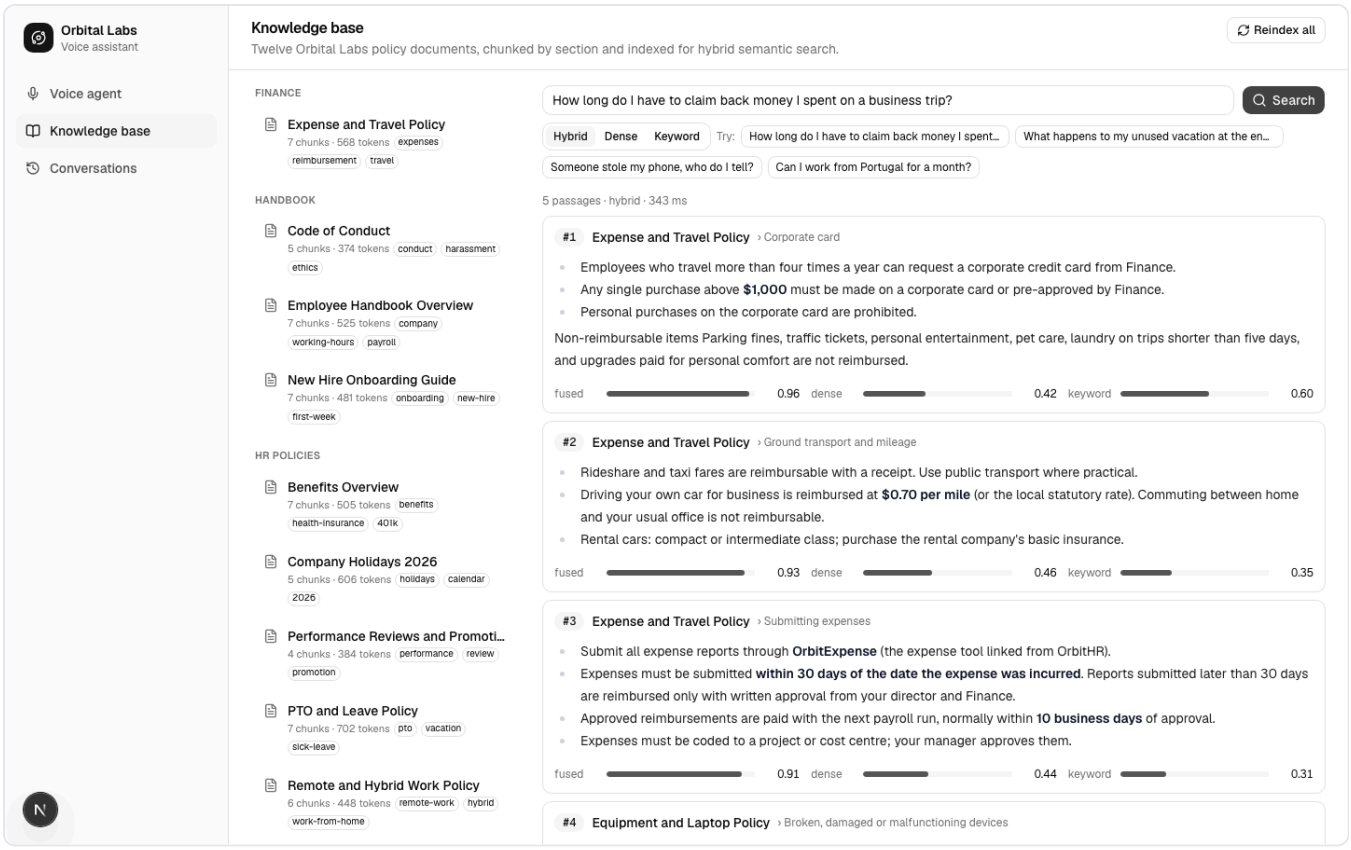


Figure 10. Knowledge base: documents by category and the search playground with fused, dense and keyword scores.

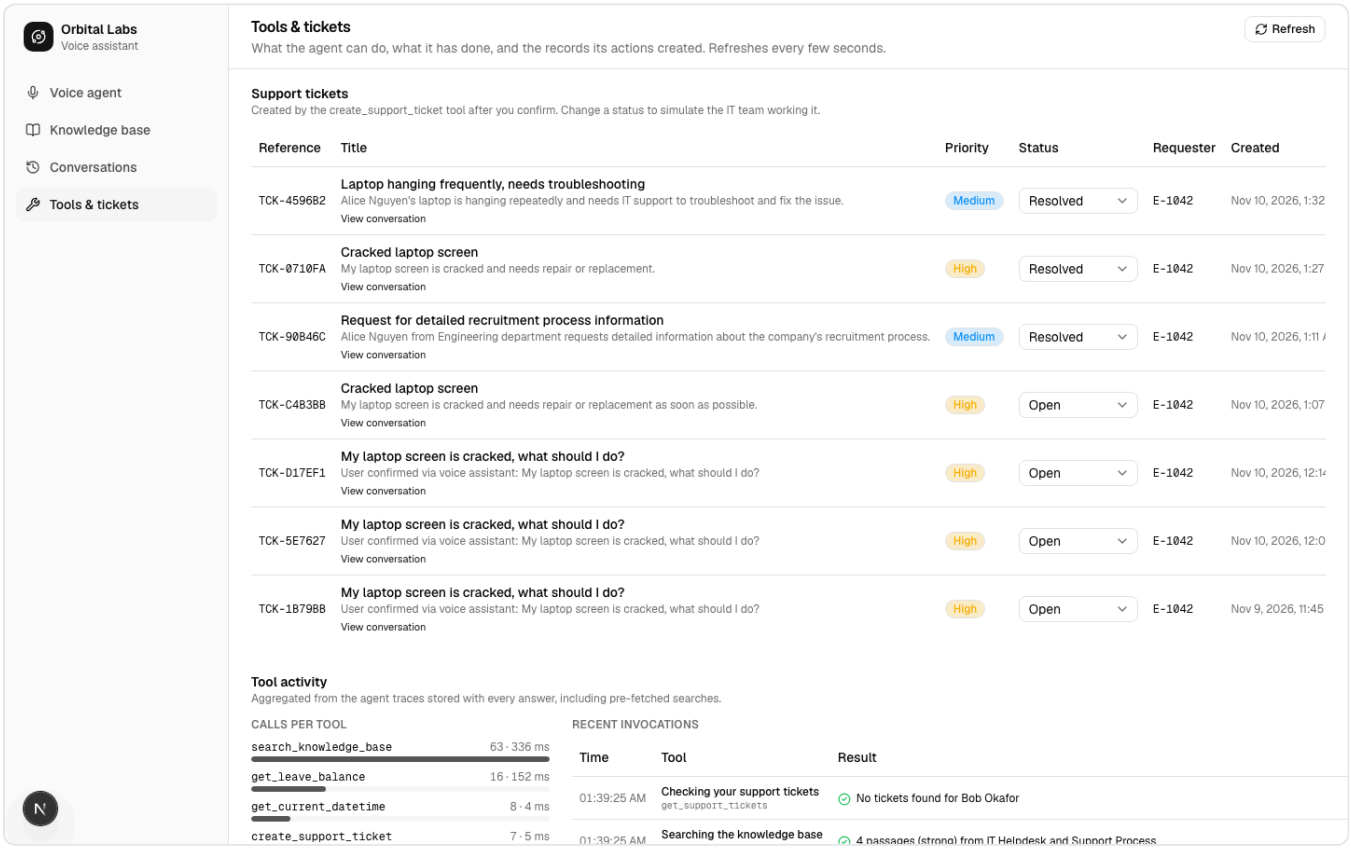


Figure 11. Tools & tickets: support tickets, tool activity, catalogue and HR balances.